

Experiment 2: Case Study

Library Management System — An Original Case Study

Atharva Jain(2420030092)

Problem Statement: Create an original real-world case study based on an application of your choice. The case study should include an introduction, user stories, a chosen software development process model, suitable programming language and technologies, an implementation strategy, an evaluation strategy, at least two UML diagrams, and a conclusion.

Real-World Case Study: Library Management System (LMS) — a digital system for managing a library's catalog, memberships, circulation, and fines.

1. Introduction

What is a Library Management System?

A Library Management System (LMS) is a software application that automates the day-to-day operations of a library — cataloging books, managing memberships, tracking borrowing and returns, handling reservations, and calculating overdue fines. It replaces manual registers with a searchable, centralized digital record.

Core idea:

One catalog, real-time availability, and automated circulation tracking for members, librarians, and administrators.

Scope of the System

- Book cataloging and search
- Member registration and profile management
- Book issue, return, and renewal (circulation)
- Reservation and hold requests
- Fine calculation and online payment
- Usage and inventory reporting for administrators

Functional Requirements

Member

- Search the catalog and reserve or borrow books
- Renew loans and view borrowing history
- Pay fines online

Librarian

- Issue and process the return of books
- Manage catalog entries and member records

Administrator

- Manage staff accounts and access rights
- Configure system settings and fine policies
- Generate usage and inventory reports

2. User Stories

A User Story is a short, simple description of a software feature written from the end user's perspective, in the format: "As a [user], I want [functionality] so that [benefit]."

Member

User ID	User Story	Priority
US-01	As a member, I want to search the catalog by title, author, or category so that I can quickly find books I'm interested in.	High
US-02	As a member, I want to reserve a book that is currently unavailable so that I am notified when it is returned.	Medium
US-03	As a member, I want to borrow a book so that I can read it outside the library.	High
US-04	As a member, I want to renew a borrowed book online so that I don't have to visit the library in person.	Medium
US-05	As a member, I want to view my borrowing history so that I can track what I've read and returned.	Low
US-06	As a member, I want to pay any pending fines online so that my account stays in good standing.	High

Librarian

User ID	User Story	Priority
US-07	As a librarian, I want to issue a book to a member so that the loan is recorded accurately.	High
US-08	As a librarian, I want to process a book return so that the book's availability status is updated.	High
US-09	As a librarian, I want to add and update catalog entries so that the collection stays current.	High
US-10	As a librarian, I want to manage member records so that membership details remain accurate.	Medium

Administrator

User ID	User Story	Priority
US-11	As an administrator, I want to generate usage and inventory reports so that I can plan acquisitions and budgets.	Medium
US-12	As an administrator, I want to manage staff accounts and access rights so that system access stays secure.	High
US-13	As an administrator, I want to configure fine rates and loan durations so that library policies can be enforced consistently.	Medium

3. Software Development Process Model

Incremental Model

Reason for Selection

- Library operations (cataloging, circulation, fines) are well-understood and stable, so requirements can be defined fairly completely upfront — unlike domains with fast-changing needs.
- The system naturally decomposes into independent, working modules (catalog, membership, circulation, reservations, reporting).
- Each increment delivers a usable piece of functionality that library staff can start using and giving feedback on before the next increment begins.
- Risk is reduced because core functionality (catalog and circulation) is built and stabilized early, before less critical features (reporting, notifications).
- Easier to plan and estimate than a purely adaptive model, since the overall requirement set does not change drastically once defined.

Increment Plan

Increment	Delivered Functionality
Increment 1	Catalog Management — add, update, and search books
Increment 2	Membership Management — registration and member profiles
Increment 3	Circulation Module — issue, return, and renew books
Increment 4	Reservation & Fine Module — holds and overdue fine calculation
Increment 5	Reporting & Admin Dashboard — usage and inventory reports
Increment 6	System-wide Testing & Deployment

4. Programming Language and Technologies

Backend

Python (Django Framework)

- Rapid development through Django's built-in ORM and admin interface
- Well suited for CRUD-heavy, data-driven applications like a library catalog
- Strong support for authentication, permissions, and role-based access (Member / Librarian / Administrator)
- Large ecosystem of libraries for reporting, PDF generation, and email notifications

Frontend

- HTML5
- CSS3
- Bootstrap
- JavaScript

Database

- PostgreSQL

Other Technologies

- Django REST Framework (for API endpoints)
- Barcode / RFID scanning integration for physical book check-in and check-out
- Celery + Redis (for scheduled overdue-reminder emails)
- Git / GitHub
- Docker
- Nginx (deployment web server)

5. Implementation Strategy

Phase 1 – Requirement Analysis

- Gather requirements from librarians, members, and administrators.
- Prepare the Software Requirement Specification (SRS).
- Identify core entities: Book, Member, Librarian, Loan, Reservation, Fine.

Phase 2 – System Design

- Design the relational database schema.
- Design UI wireframes for search, circulation, and dashboard screens.
- Prepare UML diagrams (use case and class diagrams).

Phase 3 – Development (per increment)

- Build the catalog module with search and filtering.
- Build member registration and profile management.
- Build circulation logic for issue, return, and renewal.
- Build reservation queueing and automatic fine calculation.
- Build the admin reporting dashboard.

Phase 4 – Testing

Unit Testing

- Book search and filtering
- Fine calculation logic

Integration Testing

- Circulation + Fine module
- Reservation + Catalog availability

System Testing

- End-to-end library workflow testing

User Acceptance Testing (UAT)

- Validation with librarians and sample members before go-live.

Phase 5 – Deployment

- Docker Container
- Cloud VM (AWS EC2 / equivalent)
- PostgreSQL Database
- Nginx Web Server

Phase 6 – Maintenance

- Fix defects, update fine policies, and improve performance as usage grows.

6. Evaluation Strategy

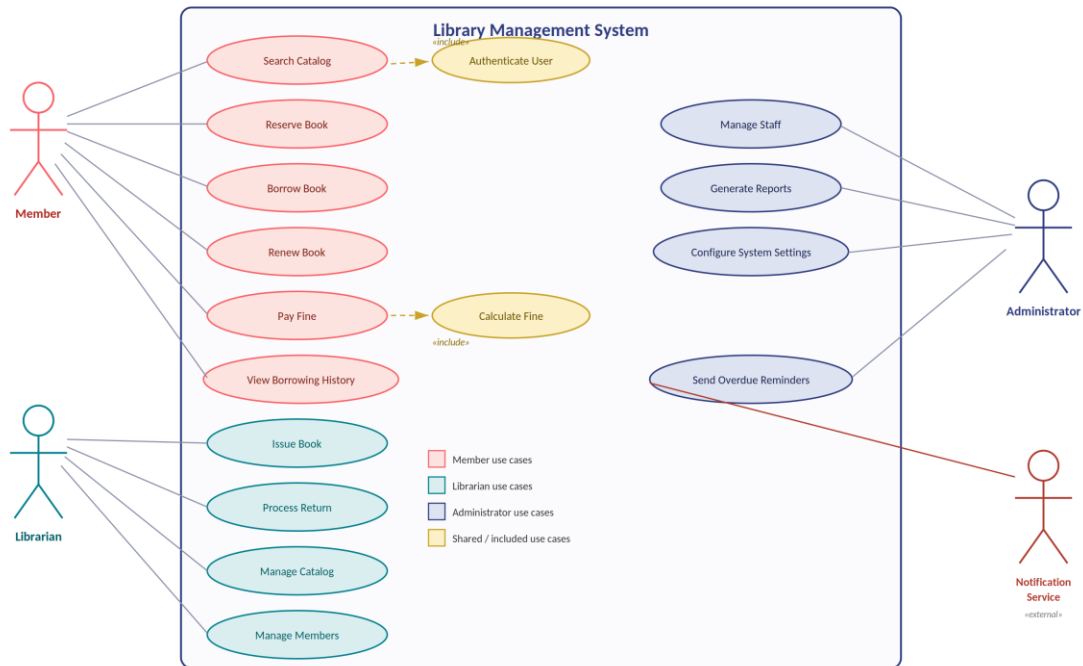
Evaluation Parameter	Method
Functional Accuracy	Requirement verification against user stories
Performance	Load testing catalog search using JMeter
Security	Authentication, role-based access, and penetration testing
Reliability	Continuous monitoring, backup and recovery checks
Usability	Feedback survey from librarians and members
Scalability	Concurrent user and large-catalog stress testing
Maintainability	Code review and documentation quality checks

Expected Outcomes

- Centralized, searchable catalog replacing manual registers.
- Faster issue and return processing through barcode-based circulation.
- Accurate, automated fine calculation and online payment.
- Reduced lost or overdue books through automated reminders.
- Reliable usage and inventory data for acquisition planning.
- Scalable architecture that supports a growing catalog and membership base.

7. UML Diagrams

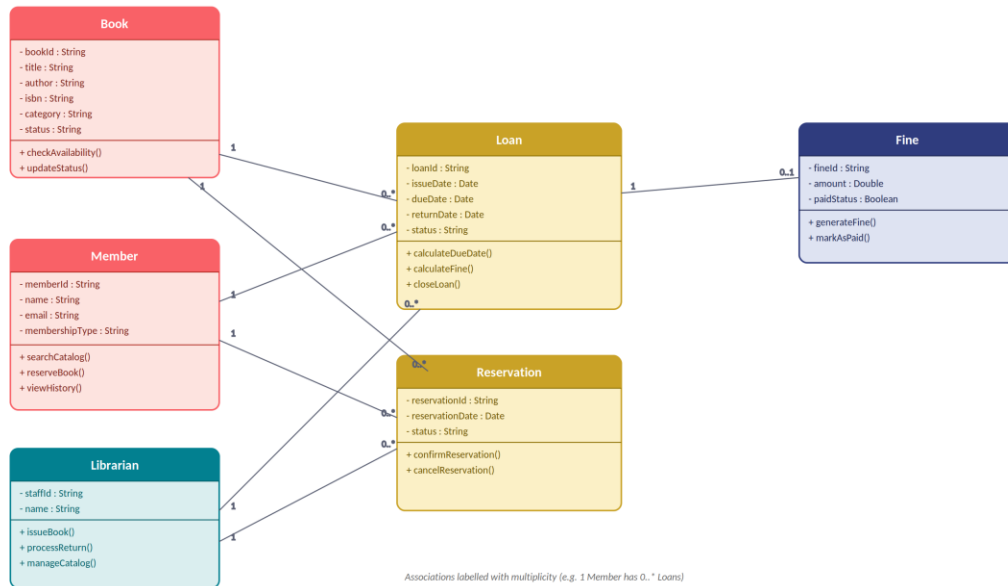
7.1 Use Case Diagram



The diagram shows the three actors of the Library Management System — Member, Librarian, and Administrator — along with a Notification Service as an external system actor. “Authenticate User” and “Calculate Fine” are shared use cases pulled in through «include» relationships, since login is a prerequisite for role-specific actions and fine calculation is required before a payment can be made.

7.2 Class Diagram

Class Diagram — Library Management System



The class diagram models the core entities of the system: Book, Member, Librarian, Loan, Reservation, and Fine. A Member can have multiple Loans and Reservations; a Librarian processes Loans and Reservations; and each Loan may generate at most one Fine, capturing the key business rules of the circulation process.

8. Conclusion

The Library Management System digitizes and streamlines core library operations — cataloging, circulation, reservations, and fine management — for members, librarians, and administrators alike.

User Stories

Captured the real needs of members, librarians, and administrators as the basis for system requirements.

Incremental Model

Provided a structured, low-risk path to deliver working modules progressively, suited to the library domain's stable requirements.

Python / Django Stack

Offered a fast, secure, and maintainable implementation platform for a data-driven, CRUD-heavy application.

Structured Strategy

Ensured systematic design, development, testing, and evaluation, resulting in a reliable and scalable system ready for real-world library use.